

1. You are given **N** signal events from a sensor stream. Each event line contains:

label amp dur

label: name of the event

amp: an integer denoting amplitude

dur: an integer denoting duration

Your task is to:

Store each event as a tuple (label, amp, dur) inside a list. Then build a dictionary

summary where for each label you store:

a. total_dur = sum of dur

b. total_energy = sum of (amp * amp * dur)

For each distinct label, print (**label total_dur total_energy**) sorted by label ascending. Also, print the label with maximum total energy (if tie, choose lexicographically smaller label).

Input	Output
6 spike 3 3 noise 1 5 spike 2 3 pulse 4 1 noise 2 2 pulse 3 2	noise 7 13 pulse 3 34 spike 6 39 TOP spike 39

2. You are given marks of **S** students in **M** subjects as a matrix **A (shape S x M)**. You are also given a vector **max_marks (length M)** containing maximum marks for each subject. You must perform several operations and print results.

Normalize marks

Compute percentage matrix:

$$P[i, j] = 100 * (A[i, j] / \text{max_marks}[j])$$

Student summary

For each student *i*, compute: avg[*i*] and best_sub[*i*]

Subject summary

For each subject *j*, compute: sub_mean[*j*] and top_student[*j*]

Best K Summary

Sort students by their average percentage and print the best K student IDs.

Unique grade Count

Convert each percentage to a letter grade using:

A	$P \geq 80$
B	$60 \leq P < 80$
C	$40 \leq P < 60$
D	$P < 40$

Count how many of each grade occur in the entire matrix (all students $\times$ all subjects).

3. Create a Student class to manage student information in your university system. Each student has a name, ID, and a list of course grades.

Class Attribute: University Name

Instance Attributes: name, student ID, grades

Methods:

Define init method

add_grade(self, grade): Add a grade to the grades list

get_average(self): Calculate and return the average of all grades

get_letter_grade(self): Return letter grade based on average marks

Define the str method

Take 5 students, 3 subjects. Provide input and test yourself.

4. You are analyzing the monthly sales data of three products (Product A, B, and C) for the year 2024. Create visualizations to compare their performance.

Monthly data

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

Sales data (in thousands)

product_a = np.array([23, 25, 28, 30, 35, 38, 42, 45, 48, 50, 53, 55])

product_b = np.array([30, 32, 31, 33, 35, 37, 39, 41, 43, 45, 47, 50])

product_c = np.array([15, 18, 22, 25, 28, 32, 35, 38, 42, 45, 48, 52])

Requirements

Create a 2×2 subplot figure showing:

Plot 1 (Top-Left): Line Plot

Plot all three products on the same graph

Product A: blue solid line

Product B: green dashed line

Product C: red dotted line

Title: "Monthly Sales Trends 2024"

X-label: "Month"

Y-label: "Sales (in thousands)"

Add legend

Enable grid

Plot 2 (Top-Right): Bar Chart

Show sales for all three products in December only

Use different colors for each product

Title: "December Sales Comparison"

X-label: "Product"

Y-label: "Sales (in thousands)"

Enable grid on y-axis

Plot 3 (Bottom-Left): Scatter Plot

Plot Product A sales as scatter points

Color: blue, marker size: 50

Title: "Product A Sales Distribution"

X-label: "Month"

Y-label: "Sales (in thousands)"

Enable grid

Plot 4 (Bottom-Right): Stacked Bar Chart

Show total sales per quarter

Quarter 1: Jan-Mar (sum of 3 months)

Quarter 2: Apr-Jun

Quarter 3: Jul-Sep

Quarter 4: Oct-Dec

Stack all three products

Title: "Quarterly Sales (Stacked)"

X-label: "Quarter"

Y-label: "Total Sales (in thousands)"

Add legend

Enable grid on y-axis

Figure Settings

Figure size: (14, 10)

Main title: "2024 Product Sales Analysis"

Use plt.tight_layout()

Expected Output

